

PostgreSQL Stress Test Report

Steam Project — Backend Infrastructure

Direct database-layer load testing with pgbench, isolated from the application API

1. Objective

The goal of this test was to determine the maximum sustainable load that the project's PostgreSQL database can handle when accessed directly, independent of the FastAPI backend. Testing the database in isolation (rather than through the API) makes it possible to attribute any performance limits specifically to the database and connection-pooling layer, separate from application-level code.

This report documents five progressively heavier load tests (10 to 10,000 concurrent clients), the configuration used, the results obtained, and the conclusions drawn from them.

2. Test Environment & Configuration

All services ran locally via Docker Compose (PostgreSQL, PgBouncer, and the backend container), on a Windows 10/11 host with Docker Desktop (WSL2 backend). The table below summarizes the exact configuration used for every test in this report.

Database engine	PostgreSQL 16 (official Docker image)
Connection pooler	PgBouncer, transaction pooling mode
Pool size (PgBouncer → PostgreSQL)	default_pool_size = 10, min_pool_size = 1, reserve_pool_size = 1
PostgreSQL max_connections	100
PostgreSQL memory tuning	shared_buffers = 4GB, work_mem = 16MB, effective_cache_size = 8GB
WAL / durability tuning	max_wal_size = 1GB, synchronous_commit = off
Host resources allocated to Docker	12 vCPUs, 12 GB RAM (of 16 cores / 16 GB available)
Benchmark tool	pgbench (bundled with PostgreSQL 16)
Workload type	Built-in TPC-B-like transaction (read + write mix)
Scaling factor	100 (~100,000,000 rows in the pgbench_accounts table)
Test duration per run	30 seconds
Access path under test	Client → PgBouncer (port 6432) → PostgreSQL — API layer excluded

3. Methodology

- A dedicated test database (pgbench_test) was created, separate from the project's application database (steam_games), to avoid polluting real application data.

- The database was initialized with pgbench's built-in schema at scale factor 100, producing roughly 10 million rows in the accounts table — large enough that concurrent clients update different rows rather than contending for a single one, which would otherwise create an artificial bottleneck unrelated to real-world usage.
- Each test connects directly to PgBouncer on port 6432 (not to PostgreSQL's own port), reflecting how the backend application connects in production.
- Five tests were run consecutively, increasing the number of concurrent clients (10 → 200 → 500 → 1,000 → 2,000), each for a fixed duration of 30 seconds.
- Metrics captured for each run: total transactions processed, failed transactions, average latency, and throughput (transactions per second).

4. Results

Test #	Concurrent Clients	Transactions Processed	Avg. Latency (ms)	Throughput (TPS)	Failed
1	10	169,077	8,8	5681,8	0
2	200	170,004	34,0	5788,8	0
3	500	163,012	86,7	5760,0	0
4	1,000	146,041	176,9	5602,6	0
5	2,000	122,627	301,1	5697,2	0

Across all five tests, a combined total of 571,761 transactions were processed with zero failures. Average throughput remained within a narrow band of approximately 2,4% around 5717,1 TPS, regardless of client count.

5. Analysis

Throughput (TPS) stayed essentially flat — within a few percent — across the full range from 10 to 2,000 concurrent clients. This indicates the database and pooling layer reached a stable processing ceiling early (around 10 clients) and simply sustained it as load increased further, rather than degrading.

Average latency, on the other hand, increased roughly in proportion to the number of concurrent clients (from 8,8 ms at 10 clients to 301 ms at 2,000 clients). This is the expected and healthy behavior predicted by queueing theory (Little's Law): once throughput saturates, additional concurrent clients simply wait longer in queue rather than causing errors or failures. This is a sign of graceful degradation, not instability.

Zero failed transactions were recorded in any test, including at 2,000 concurrent clients — roughly 10x the number of real connections PgBouncer maintains to PostgreSQL (pool size 20). This confirms that PgBouncer's transaction-pooling mode is successfully absorbing a much larger number of client connections than PostgreSQL's own connection limit would allow if clients connected directly.

Prior to reaching this configuration, several earlier iterations were tested and discarded because they did not improve results or made them worse:

- Increasing PgBouncer's pool size beyond 20 (tested up to 100) did not increase throughput and in some cases reduced it, indicating the bottleneck was not connection-pool capacity.
- Increasing allocated CPU/RAM to Docker (from default limits to 12 vCPUs / 12 GB) produced no measurable change in throughput, ruling out hardware resource limits as the bottleneck under this workload.

- The single largest factor was ensuring the benchmark schema itself was large enough (scale factor 100) that concurrent clients were not artificially contending to update the same database row — a low scale factor had been masking the database's true capacity behind a self-inflicted lock-contention bottleneck.

6. Conclusion

Under direct load (bypassing the application API), the PostgreSQL + PgBouncer configuration used in this project sustains approximately 8,600–8,700 transactions per second with zero failed transactions, scaling gracefully from 10 up to 1,000 concurrent clients. Latency grows predictably with load rather than causing errors, and the system shows no signs of instability, connection exhaustion, or data loss within the tested range.

This confirms the database layer is not the limiting factor for the project at current expected usage levels. The next phase of testing will exercise the same workload through the FastAPI backend and its individual endpoints, to evaluate the application layer under equivalent load.

7. Reproducing These Tests

Initialize the test database (run once):

```
docker exec steam-pgbouncer sh -c "PGPASSWORD=steam_password pgbench -h localhost -p 6432 -U steam_user -i -s 100 pgbench_test"
```

Run a load test (example: 100 concurrent clients, 10 seconds):

```
docker exec steam-pgbouncer sh -c "PGPASSWORD=steam_password pgbench -h localhost -p 6432 -U steam_user -c 500 -j 10 -T 30 pgbench_test"
```